

Guía práctica: Simulación de cargadores EVSE con OCPP y Django

Resumen ejecutivo

Esta guía condensada explica por qué y cómo sustituir un simple *flag* en tu base de datos por una **simulación completa de puntos de carga** que hablan OCPP. Configurarás cargadores virtuales que reportan estados reales (Available, Reserved, Charging...), enviarás comandos ([ReserveNow](#), [ChangeAvailability](#)) desde Django y ensayarás expiraciones de 30 minutos, fallos de red y métricas de consumo. Así validarás la lógica de reserva y bloqueo **antes** de conectar hardware físico.

1. ¿Por qué simular con OCPP real y no con un simple campo en la BD?

- Reproduce **exactamente** los estados oficiales de OCPP (Available, Reserved, Charging, Unavailable, Faulted).
 - Permite probar escenarios de red: heartbeats perdidos, reconexiones, latencia variable.
 - Facilita la futura certificación con la herramienta OCTT de la Open Charge Alliance.
-

2. Herramientas de simulación disponibles

OCPP	Proyecto	Cómo se usa	Ventajas
1.6 JSON	mobilityhouse/ocpp (Python)	100 líneas de script por conector	Mismo código que usarás en producción
1.6 JSON	ocpp_charging_point_simulator	<code>python</code> <code>ocpp_16_charge_point_sim.py</code> <code>ws://servidor:9000/</code> <code>CPMS CP-42</code>	Listo en 2 min; incluye comandos y heartbeats

1.6 / 2.0.1	EVERest demo (Docker)	<code>docker compose up</code>	Emula EV + EVSE + OCPP + métricas energía
1.6 JSON	SAP Charging Station Simulator (Node)	Scripts Node para cientos de cargadores	Gran escalabilidad

Consejo rápido: si sólo necesitas 2-3 cargadores, lanza varios procesos del script *mobilityhouse/ocpp*:

```
python cp.py ws://localhost:9000/CPMS CP-001 &
python cp.py ws://localhost:9000/CPMS CP-002 &
```

3. Conectar los simuladores a tu backend Django

WebSocket endpoint (Channels)

```
# routing.py
websocket_urlpatterns = [
    re_path(r'^CPMS$', CentralSystem.as_asgi()),
]
```

1.

2. **CentralSystem consumer**

Hereda de `ChargePoint` del paquete `ocpp` y sobrescribe `on_connect`, `on_message`, etc.

Reserva real (30 min)

```
from ocpp.v16 import call
from datetime import datetime, timedelta, timezone

expiry = (datetime.now(timezone.utc) + timedelta(minutes=30)).isoformat()
response = await cp.call(call.ReserveNowPayload(
    connector_id=1,
    id_tag=user.id_tag,
    expiry_date=expiry,
    reservation_id=reservation_id,
))
```

3. El simulador responde `Accepted` y envía `StatusNotification(Reserved)`.

4. Mantener la ventana de 30 min y bloquear otras reservas

- **En el cargador:** el simulador cambia a *Reserved* hasta que se inicia carga o vence `expiry_date`.
 - **En la BD:** guarda `reservation_id`, `connector_id` y `expires_at`. Impide con un *constraint* que un usuario tenga más de una reserva activa.
 - **Liberación automática:** tarea Celery cada minuto: si `expires_at < now()`, envía `CancelReservation` y marca Available.
 - **UI:** escucha `StatusNotification` y muestra cuenta atrás al usuario.
-

5. Hacer que “parezca de verdad”

Comportamiento	Implementación
Heartbeats	Enviar <code>HeartBeat</code> cada 30 s; si no llega, marcar <i>Offline</i> .
Consumo de energía	En tablas <code>MeterValues</code> , sumar aleatorio 0.3–0.7 kWh cada 60 s durante <i>Charging</i> .
Fallos ocasionales	Enviar <code>StatusNotification(Unavailable)</code> durante 2 min cada X horas.
Latencia	<code>await asyncio.sleep(random.uniform(0.2, 1.0))</code> antes de responder.

6. Pruebas automáticas (opcional)

Integra **OCTT** en tu pipeline CI para ejecutar los casos oficiales de reserva, indisponibilidad y métricas energéticas.

7. Próximos pasos

1. Arranca 2–3 simuladores con *mobilityhouse/ocpp* y valida tu flujo.

2. Cuando necesites tramas energéticas (ISO 15118), migra a **EVerest**.
 3. Añade Celery + señales Django para manejar expiraciones y evitar dobles reservas.
 4. Exporta métricas Prometheus o logs Graylog a partir de [StatusNotification](#).
-

Recursos útiles

- Open Charge Alliance — OCPP 1.6 Specification PDF.
 - [mobilityhouse/ocpp](#) — librería Python y ejemplos.
 - EVerest Project — stack de carga open-source.
 - OCTT — OCPP Compliance Test Tool.
-

¡Con esta guía podrás simular toda una red de cargadores y afinar tu backend antes de conectar el primer hardware real!